

Core java

Data types, typecasting, operators

```
public class DataCastOper {

    public static void main(String[] args) {
        System.out.println("hello there ");

        // data types => primitive & non-primitive

        // primitive => int, float, double, long
        // int data = 10 ;

        // int => 4 bytes , 1 bytes => 8 bit
        // long => 8 bytes ,
        // float => 4 bytes , decimal point -> 10.8 , 8.9999
        // double=> 8 bytes ,

        // char s = '1';
        // boolean isEligible = true;

        // non-primitive data type
        // String fiit = "Python full stack ";
        // String academy ="Java";

        // typecasting -> change from one data type to another

        // implicit => manages by compiler
        // int data = 10;
        // double implicit = 5.67;

        // implicit = data;

        // System.out.println(data);
        // System.out.println(implicit);
        // explicit => manages by user/developer.

        // data = (int)implicit;

        // System.out.println(data);
        // System.out.println(implicit);

        // operators => unary , binary , ternary

        // int a= 10;
        // unary operator => ++, -- (increment , decrement)
        // System.out.println(a++);
```

```

// System.out.println(++a);
// System.out.println(a--);
// System.out.println(--a);
// System.out.println(a);

// binary operator => two operand

// arithmetic , relational, logical, assignment, bitwise

// arithmetic => +, -, *, /, %
int a = 10;
int b = 10;
System.out.println(a+b);
System.out.println(a-b);
System.out.println(a*b);
System.out.println(a/b);
System.out.println(a%b); //reminder

// relational => > , < , >= , <= , != , ==
System.out.println(a>b);
// true or false,
System.out.println(a<b);
System.out.println(a>=b);
System.out.println(a<=b);
System.out.println(a==b);

// logical operator => && || !

if(true || false) {
    System.out.println("True");
}

if(true){
    System.out.println(!false);
}

}
}

```

Theory Notes

1. Data Types

- **Primitive types** → fixed size, stored directly in memory.
 - int (4 bytes), long (8 bytes), float (4 bytes), double (8 bytes), char (2 bytes), boolean (1 bit).

- **Non-primitive types** → objects, arrays, strings.

- Example: `String name = "Java";`.

2. Type Casting

- **Implicit casting (widening)** → smaller type automatically converted to larger type.

```
int x = 10;
```

```
double y = x; // int → double
```

- **Explicit casting (narrowing)** → larger type manually converted to smaller type.

```
double d = 9.8;
```

```
int i = (int) d; // double → int
```

3. Operators

- **Unary operators** → single operand (`++`, `--`).
- **Binary operators** → two operands.
 - Arithmetic: `+`, `-`, `*`, `/`, `%`.
 - Relational: `>`, `<`, `>=`, `<=`, `==`, `!=`.
 - Logical: `&&`, `||`, `!`.
 - Assignment: `=`, `+=`, `-=`, etc.
 - Bitwise: `&`, `|`, `^`, `<<`, `>>`, `~`.
- **Ternary operator** → shorthand for if-else.

```
int age = 18;
```

```
String result = (age >= 18) ? "Eligible" : "Not Eligible";
```

4. Key Points

- Java is **statically typed** → variable type must be declared.
 - Type casting is essential when working with mixed data types.
 - Operators are the building blocks for logic and arithmetic in programs.
1. Explore **ternary operator** for concise conditions.
 2. Try **bitwise operators** to understand binary-level operations.

Practice Tasks

Short Answer

1. What is the difference between implicit and explicit type casting?
2. Why is Java called statically typed?
3. What does % operator return?

Coding Exercises

1. Write a program to demonstrate implicit casting (int → double).
2. Perform explicit casting (double → int) and show the result.
3. Use unary operators to increment and decrement a number.
4. Write a program that checks if a number is divisible by 2 using %.
5. Implement a voting eligibility check using the ternary operator.
6. Demonstrate logical operators (&&, ||, !) with conditions.